Dall'approccio Modulare alla Policy End-to-End

Corso di Robotica, A.A. 2024-2025

Mattia Fornari, Luca Pugnetti

UNIVERSITÀ DEGLI STUDI DI BRESCIA

Introduzione

Robotica Autonoma vs. Deep RL

Questo progetto analizza le differenze tra due filosofie di navigazione autonoma:

1. **Approccio Modulare**
   - Decomposizione del problema in moduli.
   - Richiede modelli fisici e matematici espliciti.
2. **Deep RL**
   - Approccio unico ("End-to-End") di Reinforcement Learning.
   - L'agente impara per tentativi mediante l'algoritmo PPO, senza conoscere la fisica a priori.

Applicazione Pratica: Addestramento di un agente nell'ambiente Gymnasium **LunarLanderContinuous-v3** mediante l'algoritmo di Reinforcement Learning PPO con stable-baselines3.

L'ambiente LunarLanderContinuous-v3

LunarLanderContinuous-v3 - Gymnasium Farama

Ambiente standard di Farama **Gymnasium**, basato sul motore fisico **Box2D** e usato per valutare algoritmi di Reinforcement Learning.

Obiettivo: Atterrare un lander su una piattaforma a coordinate $(0, 0)$, arrivando con velocità quasi nulla ($v \approx 0$) e massimizzando il reward.

Le sfide fisiche per l'ambiente con impostazioni di default:

- **Gravità**: $g = -10.0$ (rende la discesa più rapida, scelta di design di gymnasium)
- **Inerzia**: assenza di attrito atmosferico (l'agente non rallenta se non frena)
- **Efficienza**: penalità per l'uso continuo dei motori
- **Ambiente statico**: assenza di vento o turbolenze (configurazione standard).

Spazio degli stati ($s \in \mathbb{R}^8$):

$$s = [x,\ y,\ v_x,\ v_y,\ \theta,\ \omega,\ c_L,\ c_R]$$

Dove: **posizione:** $x \in [-2.5, 2.5]$, $y \in [0, 2.5]$ — **velocità:** $v_x, v_y \in [-10, 10]$ — **angolo:** $\theta \in [-2\pi, 2\pi]$, $\omega \in [-10, 10]$ — **stato di contatto delle gambe con il suolo:** $c_L, c_R \in \{0, 1\}$ variabili booleane

Interazione tra agente e ambiente

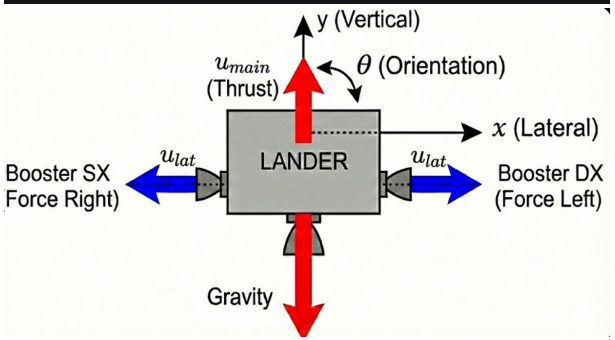
L'agente controlla il lander inviando all'ambiente un vettore continuo $\mathbf{u} \in [-1, 1]^2$:

Motore principale (u_{main}): Gestisce la spinta verticale.

- $u_{\text{main}} < 0$: motore **spento**.
- $0 \leq u_{\text{main}} \leq 1$:
- $(0, 1]$: potenza lineare dal **50% al 100%**.

Motori Laterali (u_{lat}): Controllano θ e x.

- $-0.5 < u_{\text{lat}} < 0.5$: **Dead-zone** (spenti per risparmio carburante e stabilità).
- $u_{\text{lat}} < -0.5$: si attiva il booster a
- $u_{\text{lat}} < -0.5$: Booster **SX** (spinge a destra $\rightarrow$).
- $u_{\text{lat}} > 0.5$: Booster **DX** (spinge a sinistra $\leftarrow$).



Schema delle forze applicate in base al vettore **u**.

Il reward incentiva ad avvicinarsi lentamente alla piattaforma, penalizzando l'uso eccessivo dei motori e il movimento veloce/inclinato. (Gymnasium)

Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO): algoritmo model-free, *on-policy* di Deep RL.

Architettura Actor-Critic

- **Actor** $\pi_\theta(a|s)$: rete neurale che apprende la *policy*, mappando stati in azioni.
  - Input: stato $s \in \mathbb{R}^8$
  - Output: azione $a \in \mathbb{R}^2$ (spinta motore principale e laterale)
- **Critic** $V_\phi(s)$: stima il valore atteso dello stato (return futuro). Serve per calcolare il **Vantaggio** $\hat{A}_t$ (quanto l'azione è migliore/peggiore del previsto).

Clipped Objective:

$$L^{CLIP}(\theta) = \mathbb{E}\left[\min\left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right]$$

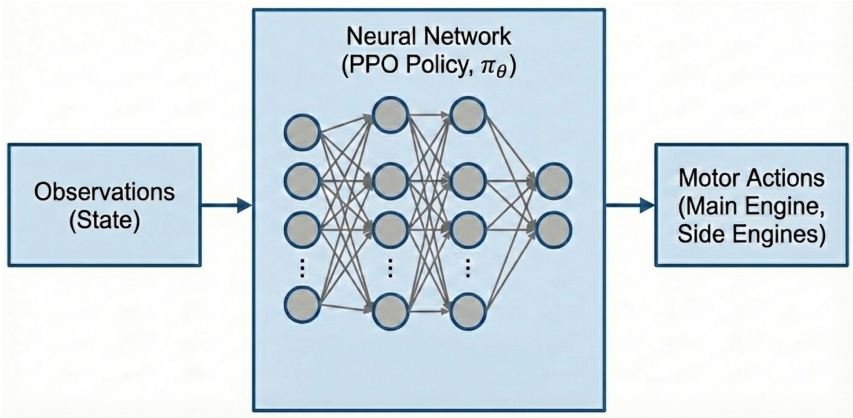
dove $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ misura il cambiamento della policy. Il **clipping** (con $\epsilon = 0.2$) limita aggiornamenti troppo grandi, garantendo stabilità nel training.

PPO Stable Baselines3

PPO approccia il problema in modo "End-to-End", mappando direttamente gli stati nelle azioni tramite Deep Learning usando i dati di interazione con l'ambiente (stati, azioni, reward).

Multi-Layer Perceptron (MLP):
PPO in utilizza **due reti neurali** che condividono i layer nascosti:

- **Input:** lo stato $s_t \in \mathbb{R}^8$
- **Shared Hidden Layers:** Estraggono feature non-lineari comuni
- **Actor Head** π_θ: output (μ_θ, σ) della policy (media e deviazione standard per azioni continue)
- **Critic Head** V_ϕ: stima il valore dello stato $V(s_t)$



Policy Stocastica: L'azione (se continua) viene campionata da una distribuzione Gaussiana $a_t \sim \mathcal{N}(\mu_\theta(s_t), \sigma)$, introducendo esplorazione casuale.

PPO per LunarLanderContinuous-v3

Obiettivo: risolvere l'ambiente con un reward ≥ 200 punti. Il processo alterna due fasi distinte:

Fase 1: Rollout (raccolta dati) L'agente agisce nell'ambiente per N passi usando la policy corrente.

- Osserva lo stato s_t dall'ambiente.
- Campiona l'azione a_t
- Esegue l'azione e raccoglie (s_t, a_t, r_t, s_{t+1}).
- Calcola il valore stimato $V(s_t)$ dal Critic.

$\rightarrow$ *In questa fase i pesi della rete sono congelati.*

Fase 2: Training (Update ogni N passi) Sui dati raccolti, si aggiornano le reti per K epoche

- Si calcola il Vantaggio $\hat{A}_t$
- I dati vengono divisi in mini-batch
- Si esegue la Backpropagation **minimizzando** la funzione obiettivo L_{tot}.

$\rightarrow$ *Qui l'agente impara effettivamente.*

Funzione Obiettivo Totale (Loss):

$$L_{tot} = -L^{CLIP}(\theta) + c_1 \underbrace{L^{VF}(\theta)}_{\text{Errore del Critic}} - c_2 \underbrace{S[\pi_\theta](s)}_{\text{Bonus Entropia}}$$

Implementazione: train_lander.py

Configurazione: `stable_baselines3.PPO` con `MlpPolicy` (reti neurali fully-connected). PPO impara a bilanciare velocità, orientamento e consumo energetico.

Iperparametri chiave:

- **n_steps:** 2048 passi
- **Batch Size:** 64
- **Learning Rate:** $3 \cdot 10^{-4}$.
- **Entropy coef:** 0.01
- **Discount (γ):** 0.99
- **clip range** ϵ : 0.2

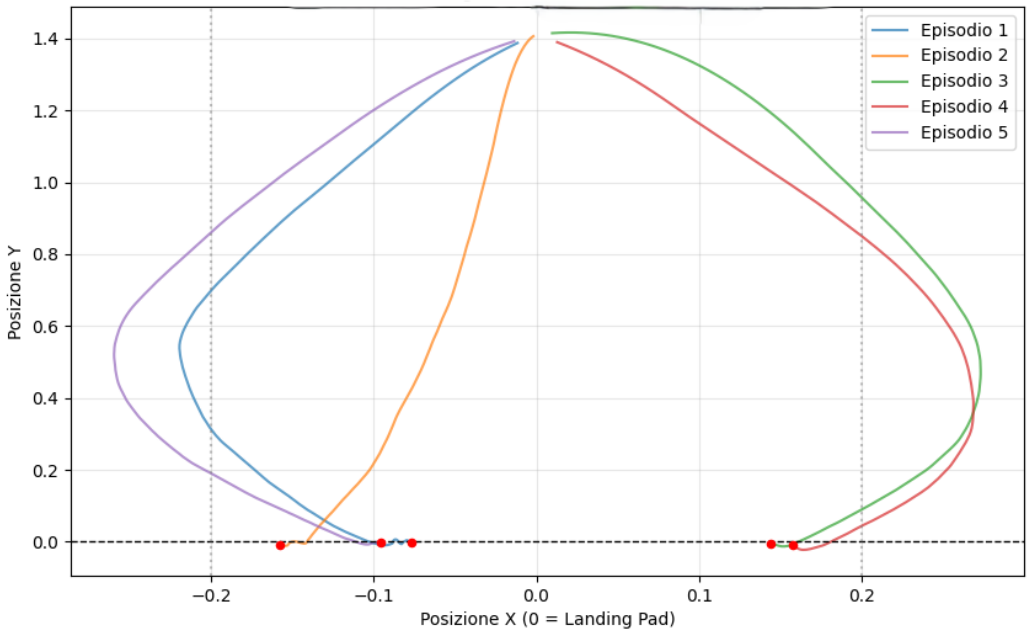
```python
# Estratto da train_lander.py
model = PPO(
    "MlpPolicy",
    env,
    learning_rate=3e-4,
    n_steps=2048,
    batch_size=64,
    n_epochs=10,
    gamma=0.99,
    gae_lambda=0.95,
    clip_range=0.2,
    ent_coef=0.01,
    device="cpu",
    verbose=1,
    tensorboard_log="./tensorboard_logs/"
)
model.learn(total_timesteps=2000000)
```

Training: 2M timesteps totali e reward definito dall'ambiente.

"Policy stocastica continua": `MlpPolicy` genera la media $\mu_\theta(s_t)$ tramite l'Actor e usa una deviazione standard appresa a parte durante il training (indipendente dallo stato). L'azione è campionata come $a_t \sim \mathcal{N}(\mu_\theta(s_t), \sigma)$.

Risultati: Analisi delle traiettorie

Abbiamo valutato la policy addestrata su 5 episodi con punti di partenza (spawn) casuali differenti.

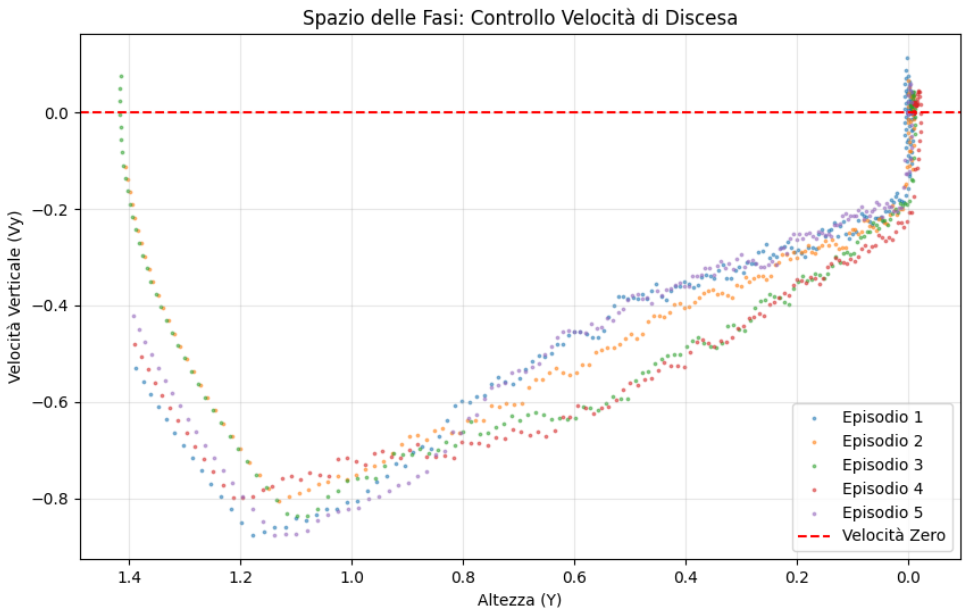


Osservazioni:

- **Convergenza**
  Indipendentemente da dove nasce il lander (sinistra, destra, alto), tutte le traiettorie convergono approsimativamente verso $(0, 0)$.
- **Generalizzazione:**
  L'agente corregge attivamente le deviazioni laterali generate dal motore principale.

Analisi nello spazio delle fasi

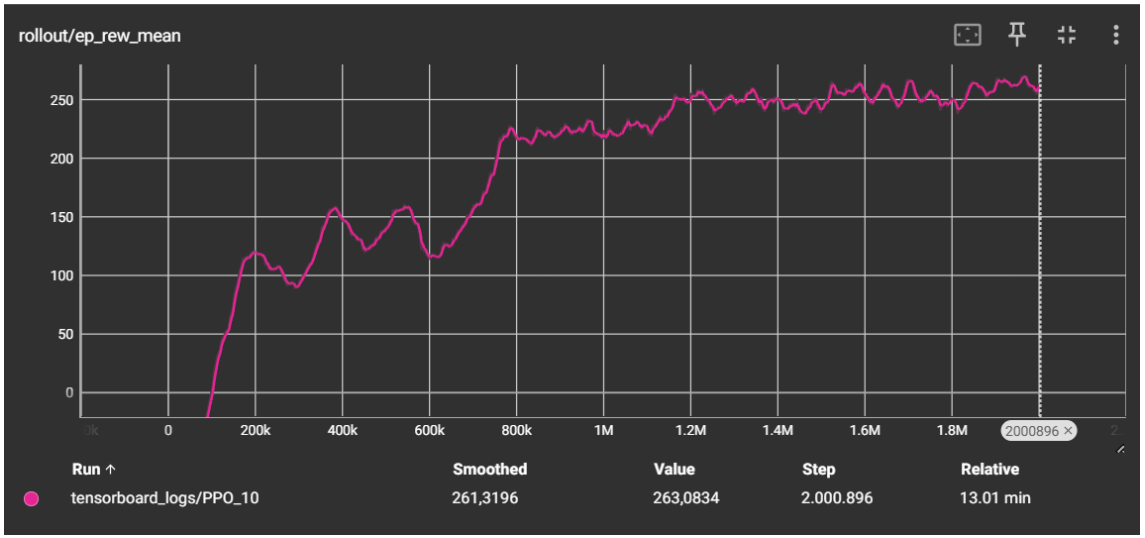
Per verificare la qualità dell'atterraggio (se sicuro e controllato), abbiamo analizzato la relazione tra (y) e (v_y).

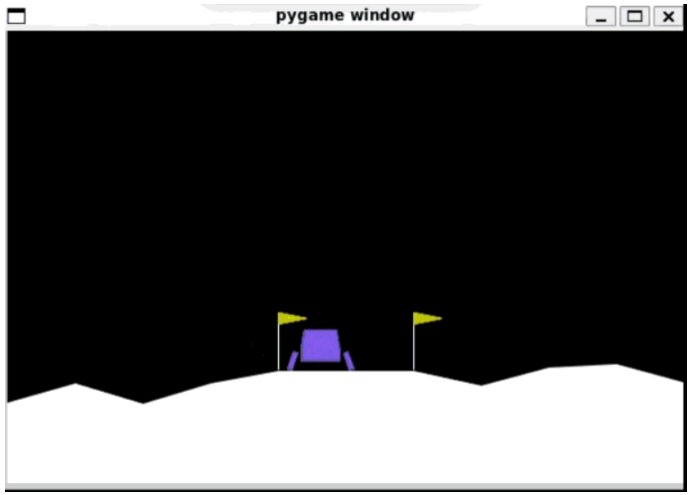


Osservazioni:

- **Decelerazione controllata:** la curva mostra che al diminuire dell'altezza $(y \to 0)$, anche la velocità tende a zero $(v_y \to 0)$.
- **Hovering finale:** il tratto verticale a $y \approx 0$ evidenzia la manovra di frenata a quota costante circa.
- **Convergenza:** l'agente impara a frenare progressivamente.

Altri risultati





Esempio episodio ottenuto con visualize_lander.py

Il grafico a sinistra mostra una rapida crescita delle performance che convergono verso un reward medio di circa 260 punti. Poiché la soglia per considerare l'ambiente "risolto" è 200, l'agente ha appreso una policy robusta.
https://github.com/mattiafornari/ppo_lunar_lander_robotics_course

Approccio modulare

Modellazione Dinamica:

Mentre il PPO apprende la fisica per tentativi, l'approccio modulare richiede che il Motion Planner e il Controller conoscano esplicitamente le equazioni del moto.

Le Equazioni Differenziali del Lander: Modellando il veicolo come un corpo rigido 2D con stato $x = [x, y, v_x, v_y, \theta, \omega]^\top$, l'evoluzione dinamica è definita da:

Dinamica Traslazionale e Rotazionale

$$\dot{x} = v_x, \quad \dot{y} = v_y$$

$$\dot{v}_x = \frac{1}{m}(F_{lat}\cos\theta - F_{main}\sin\theta)$$

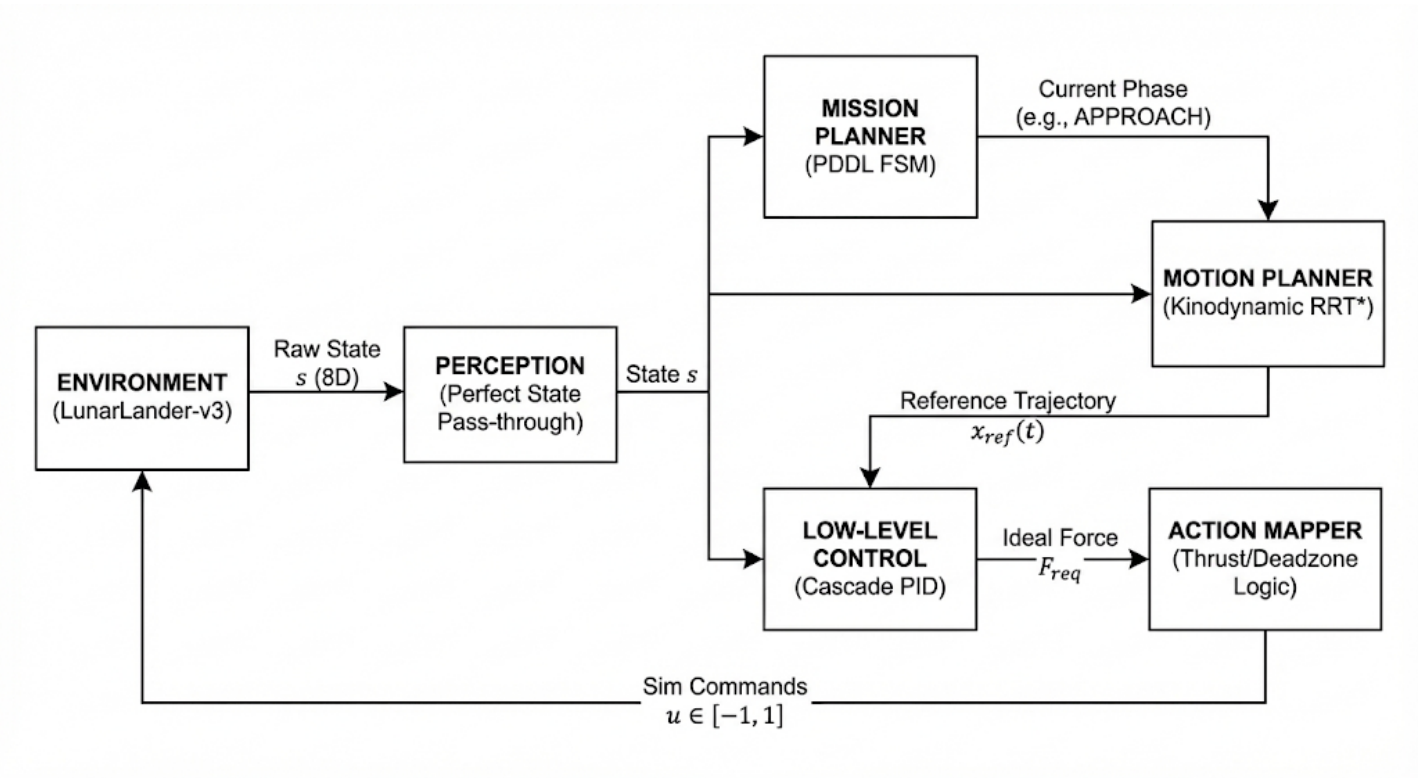
$$\dot{v}_y = \frac{1}{m}(F_{main}\cos\theta + F_{lat}\sin\theta) - g$$

$$\dot{\omega} = \frac{M}{I} - c_d\omega$$

Dove F_{main}, F_{lat} sono le forze dei motori (input di controllo trasformati), g è la gravità, m la massa, I l'inerzia e c_d lo smorzamento.

La Pipeline Modulare

Scegliere il titolo Sostituiamo la "Black Box" neurale con una catena di blocchi funzionali espliciti, dove ogni modulo ha una responsabilità unica e verificabile.



Percezione → Mission Planning → Motion Planning → Controllo → Attuazione

Perception

Il primo modulo della pipeline deve rispondere alla domanda fondamentale: *"Dove sono esattamente e come mi sto muovendo?"*

Nel Nostro Caso (Simulazione)

L'ambiente di simulazione *LunarLander* fornisce una **Full Observability** con dati perfetti e privi di rumore.

- **Tecnica:** "Pass-through".
- Lo stato $s = [x,\ y,\ v_x,\ v_y,\ \theta,\ \omega,\ c_L,\ c_R]$ viene passato direttamente ai moduli successivi senza alcuna elaborazione o filtro.

Nota: In questo scenario specifico, dove la posizione del target $(0, 0)$ *è nota a priori, non è necessario utilizzare tecniche di SLAM (Simultaneous Localization and Mapping).*

Mission Planner

Questo modulo gestisce la logica decisionale ad alto livello, determinando cosa fare in base allo stato corrente.

Il Mission Planner agisce come una Macchina a Stati Finiti (FSM). Monitora costantemente lo stato del lander (s) e decide la fase operativa corrente, inviando obiettivi specifici al Motion Planner.

Fasi Tipiche della Missione:

1. **APPROACH (Avvicinamento):** Il lander è lontano e in alto. L'obiettivo è ridurre l'errore orizzontale ($x \to 0$) mantenendo l'altitudine.
2. **DESCENT (Discesa):** Una volta sopra il target, si inizia una discesa controllata, limitando la velocità verticale (v_y) per evitare crash.
3. **LAND (Atterraggio):** Fase terminale a bassa quota. Obiettivo: $v_x \approx 0, v_y \approx 0, \theta \approx 0$.
4. **RECOVER (Emergenza):** Se l'inclinazione (θ) o la velocità angolare (ω) superano soglie critiche, si attiva una manovra prioritaria di stabilizzazione.

Landing Detection

Il passaggio allo stato finale (Mission Complete) e lo spegnimento dei motori avvengono solo al soddisfacimento congiunto di condizioni discrete e continue:

$$\text{COMPLETE} \iff (\text{Legs Contact}) \wedge (|v| < \epsilon_v) \wedge (|\theta| < \epsilon_\theta)$$

Motion Planner

Dato l'obiettivo dal Mission Planner (e.g., "scendi a $(0, 1.0)$ in fase di DESCENT"), il suo obiettivo è calcolare come arrivarci tenendo conto della fisica.

Il problema: Kinodynamic Planning

Trovare una traiettoria $x(t)$ nello spazio degli stati 6D $(x, y, v_x, v_y, \theta, \omega)$ che:

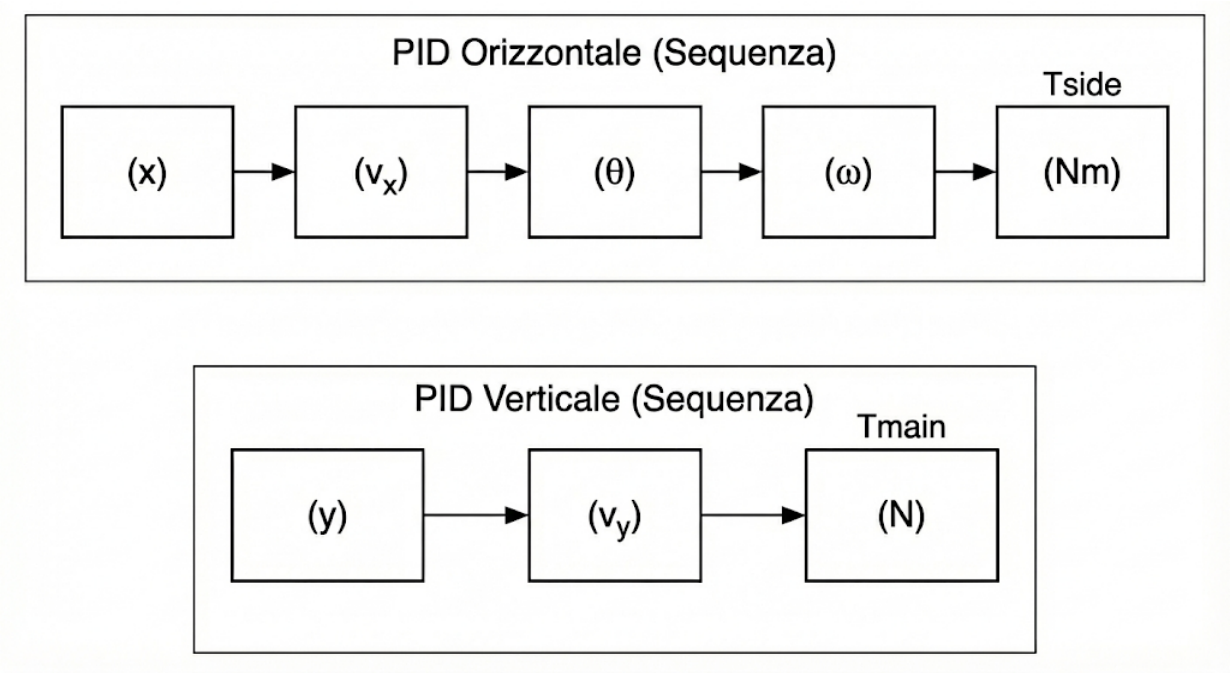
- colleghi lo stato attuale allo stato obiettivo
- rispetti i vincoli dinamici (gravità $g = -10\ m/s^2$, inerzia)
- rispetti i limiti di attuazione dell'ambiente Gymnasium)

Una possibile soluzione prevede l'utilizzo del **Kinodynamic RRT* (Rapidly-exploring Random Tree)** Estensione di RRT* che incorpora i vincoli dinamici del sistema. Esplora aggressivamente lo spazio degli stati costruendo incrementalmente un albero di traiettorie fisicamente realizzabili, radicato nello stato attuale del lander. Ad ogni iterazione propaga la dinamica con comandi ammissibili e raffina progressivamente verso l'ottimalità.

Output: traiettoria di riferimento con posizioni (x_{ref}, y_{ref}), velocità $(v_{x,ref}, v_{y,ref})$, orientamento $(\theta_{ref}, \omega_{ref})$ per il Low-Level Controller.

Low-Level Control

Dati i riferimenti di posizione (x_{ref}, y_{ref}), velocità $(v_{x,ref}, v_{y,ref})$ e orientamento $(\theta_{ref}, \omega_{ref})$ dal Motion Planner, l'obiettivo del controller è fare il Trajectory Tracking (Inseguimento della traiettoria). Deve fare in modo che lo stato reale del robot si sovrapponga allo stato desiderato ricevuto dal planner. Questo viene fatto comandando $[T_{main}, T_{side}]$. Un possibile approccio in questo caso è utilizzare un **Controllo PID in Cascata**.



Action Mapper

Il modulo di **Action Mapping** converte le grandezze fisiche (N, Nm) in comandi adimensionali $[-1, 1]$ accettati dall'ambiente, gestendo i limiti fisici.

1. Main Engine (Verticale)

Normalizzazione della forza richiesta rispetto alla spinta massima F_{max}.

$$u_{main} = \text{clip}\left(\frac{F_{req}}{F_{max}}, 0, 1\right)$$

Risultato: Throttle continuo $[0, 1]$.

2. Side Engines (Orizzontale)

Prima si normalizza la coppia rispetto a τ_{max}, poi si applica la logica discreta con **Dead-zone** ($\delta = 0.5$).

Step A: Normalizzazione

$$u_{raw} = \text{clip}\left(\frac{\tau_{req}}{\tau_{max}}, -1, 1\right)$$

Step B: Attivazione (Switch)

$$u_{side} = \begin{cases} u_{raw} & \text{se } |u_{raw}| > \delta \\ 0 & \text{se } |u_{raw}| \leq \delta \end{cases}$$

Vantaggi e svantaggi dei due metodi

Pro e Contro dell'Approccio Modulare

L'approccio modulare offre controllo totale ma a un costo elevato.

Vantaggi

- **Interpretabilità:** Se il lander fallisce, si sa esattamente quale modulo ha sbagliato.
- **Sicurezza (Safety):** È possibile imporre vincoli "duri" (Hard Constraints).

Svantaggi

- **Sforzo Ingegneristico:** Richiede tempo per lo sviluppo e conoscenza profonda della fisica e della matematica.
- **Design Complesso:** Bisogna progettare manualmente le interfacce tra i moduli.
- **Tuning Difficile:** Trovare i guadagni perfetti per il PID può essere difficile.

Pro e Contro della Policy End-to-End (PPO)

L'approccio Deep RL con PPO offre apprendimento automatico ma presenta sfide specifiche.

Vantaggi

- **Stabilità nel Training:** Il clipping previene update drastici della policy, garantendo apprendimento graduale senza "dimenticare" strategie funzionanti.
- **Esplorazione Bilanciata:** La policy stocastica Gaussiana bilancia exploration-exploitation attraverso l'entropy bonus, scoprendo strategie non intuitive.
- **Nessun Design Manuale:** Apprende automaticamente compensazioni complesse (gravità, accoppiamenti dinamici) senza derivare equazioni o tunare PID.

Svantaggi

- **Black Box:** Scarsa interpretabilità. Difficile diagnosticare fallimenti o capire matematicamente le decisioni della rete. Debug difficile.
- **Sample inefficiency:** Richiede enormi quantità di dati (500k+ tentativi) per imparare compiti semplici.
- **Sensibilità Iperparametri:** La performance dipende criticamente dal learning rate, clip range, entropy coefficient.
- **Overfitting al Simulatore:** La policy **può** sfruttare bug di Box2D che non esistono nel mondo reale. Problemi di generalizzazione con vento o gravità diverse.

Conclusioni

Grazie per l'attenzione.